arXiv:2403.14714v1 [cs.PL] 18 Mar 2024

COMPILER GENERATED FEEDBACK FOR LARGE LANGUAGE MODELS

Dejan Grubisic [1 2] **Chris Cummins** [1] **Volker Seeker** [1] **Hugh Leather** [1]

ABSTRACT

We introduce a novel paradigm in compiler optimization powered by Large Language Models with compiler feedback to optimize the code size of LLVM assembly. The model takes unoptimized LLVM IR as input and produces optimized IR, the best optimization passes, and instruction counts of both unoptimized and optimized IRs. Then we compile the input with generated optimization passes and evaluate if the predicted instruction count is correct, generated IR is compilable, and corresponds to compiled code. We provide this feedback back to LLM and give it another chance to optimize code. This approach adds an extra 0.53% improvement over -Oz to the original model. Even though, adding more information with feedback seems intuitive, simple sampling techniques achieve much higher performance given 10 or more samples.

1 INTRODUCTION

Large Language Models (LLMs) have proven their ability in the software engineering domain to generate the code and documentation (Li et al., 2023; Allal et al., 2023), translate code between programming languages (Lachaux et al., 2020; Armengol-Estapé & O'Boyle, 2021), write unit-tests (Deng et al., 2023; Schäfer et al., 2023), as well as detect and fix bugs (Ahmad et al., 2023; Xia et al., 2023). The availability of large open-source code datasets (Da Silva et al., 2021; Armengol-Estapé et al., 2022) and Github enabled models such as CodeLlama (Rozière et al., 2023), ChatGPT (OpenAI, 2023), and Codex (Chen et al., 2021) to develop a statistical understanding of various languages improving significantly the coding experience. Some of the models such as AlphaCode (Li et al., 2022) are pretrained on competitive programming tasks which enables the model to optimize code on the source level for several languages.

Motivated by those successes we decided to embrace the use of LLMs further in compilers and optimize code on the level of LLVM IR. In contrast to optimizing source code, where LLM should provide suggestions and a programmer should accept it or not, in compilers, LLM is responsible for making the final decision. As a consequence, LLM is not allowed to change semantics in any case. To guarantee that syntax is not changed, we use LLM to suggest optimization strategies for the given code and direct the compiler to perform optimization passes. This way we can get the best of both worlds by navigating complex optimization space with LLM while keeping the code provably correct.

[1]Meta AI, Menlo Park, CA, USA [2]Rice University, Houston, TX, USA. Correspondence to: Dejan Grubisic <dx4@rice.edu>.

Previous research in machine learning-driven code optimization has adopted various techniques, ranging from manually crafted features (Trofin et al., 2021; Wang & O'Boyle, 2018; Leather & Cummins, 2020) to sophisticated graph neural networks (GNNs) (Liang et al., 2023; Cummins et al., 2021). However, a common limitation across these approaches is that the representation of the input program provided to the machine learning model remains incomplete, resulting in some loss of critical information. For instance, in the case of MLGO (Trofin et al., 2021), it leverages numerical features to offer insights into function inlining but falls short in faithfully reconstructing aspects such as the call graph and control flow. Similarly, PrograML (Cummins et al., 2021) employs graph-based program representations for GNNs, yet it overlooks essential details like constant values and certain types of information, hindering the faithful reproduction of program instructions.

To overcome this problem Cummins et. all (2023) proposed LLMs for tuning LLVM optimization passes to reduce code size. By using LLVM IR code directly as an input, this approach demonstrated remarkable code reasoning abilities, outperforming default LLVM *-Oz* optimization by 2.87% while long-run autotuner used to generate data for LLM achieved 5% improvement. Instead of generating just optimization passes the model also predicts the instruction count of source and target code and optimized IR which significantly increases the performance.

We extend (Cummins et al., 2023) further by evaluating the consistency of model generation by the compiler and providing feedback to the model. For each generation of the original model, we evaluate if the generated pass list is valid and if predicted instruction counts are correct and calculate bleu score (Papineni et al., 2002) between the generated

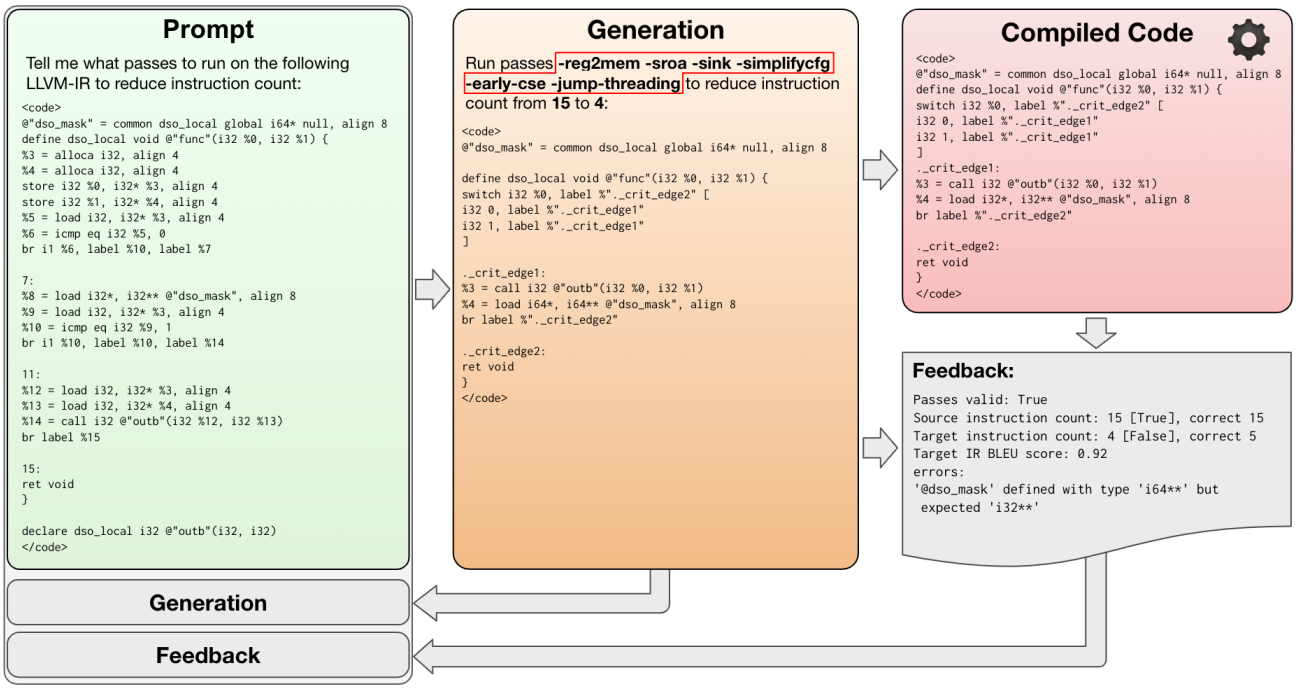


Figure 1. Feedback-directed model. First, we ask in the prompt LLM to optimize the instruction count of the given IR, then LLM generates the best optimization passes, instruction counts for starting and generated IR, and generated IR itself. Next, we compile the generated pass list and create feedback by checking if the generated pass list is valid, evaluating instruction counts, examining if the generated IR contains compilation errors, and calculating the BLEU score between the generated IR and the compiled IR. If some of the parameters of the feedback is problematic, we extend the original prompt with generation, compiled code, and feedback and ask it to try again.

code and code we get by compiling the generated pass list. Additionally, we provide generated and compiled code in the feedback.

We make the following contributions in this paper:

- We present 3 compiler-generated feedback models for LLMs (Section 2)
- We evaluate 3 sampling methods with feedback (Section 4.2)
- We evaluate iterative feedback generation (Section 5.1)

With the Feedback method, we were able to increase the performance from 2.87% to 3.4% over the *-Oz* with a single shot while failing to improve the performance of the original model when using sampling with more than 10 samples. Additionally, the Feedback method doesn't improve the performance of the original model when it is allowed to iterate 5 steps, while the original model uses sampling with the same number of inferences.

2 Feedback-directed LLMs

The performance of LLMs improves significantly when they are allowed to generate a series of reasoning steps to get to the final solution (Wei et al., 2022b). This kind of behavior is particularly true for complex problems such as arithmetic, symbolic reasoning, and code generation. Additionally, increasing stochasticity and generating multiple different solutions, leads to superior performance. In both cases, the model often generates misguided solutions that impede their ability to reason correctly. Being able to get feedback on its generation and fix its errors could enable LLM to take one step further toward coming to a favorable solution.

We explore the model shown in Figure 1 as a three-step process. In the first step, the model starts from the prompt that contains only unoptimized IR and generates an optimization pass list, instruction count, and optimized IR itself. In the second step, we derive available metrics from generation with the help of the compiler and construct feedback. The feedback purpose is to quantify the consistency of generation and to point out where the internal model of the LLM diverges from the actual compiled IR. In the third step, we provide feedback to the model and give it a second chance.

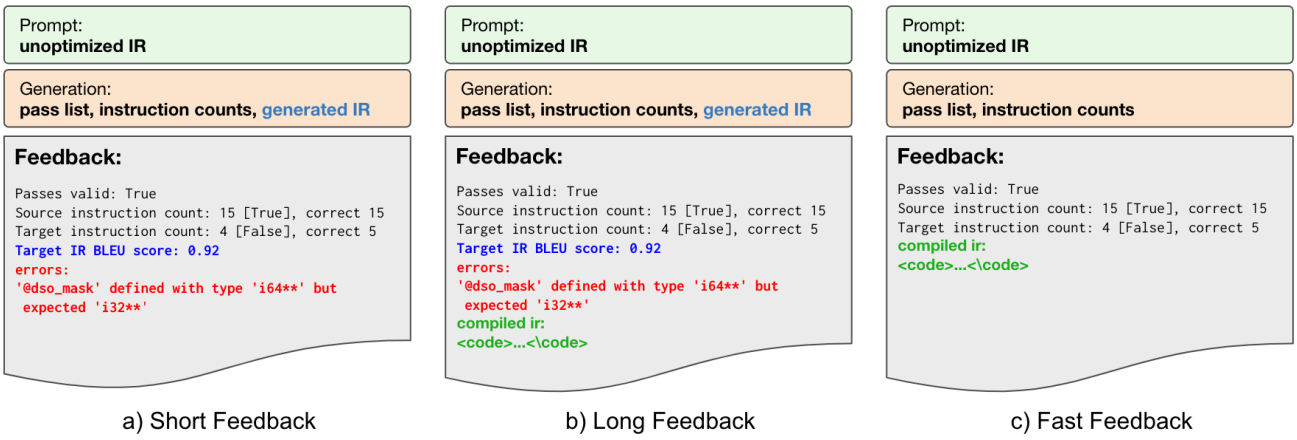


Figure 2. Prompt structure of Feedback models. Short Feedback is the smallest in size and extends the prompt with just calculated metrics and error messages. Long Feedback contains the most information including compiled IR. Fast Feedback is the fastest to generate since it doesn't need the generation of IR to be calculated.

To construct a feedback we evaluate if the generated pass list is valid, then we compile source IR with the generated pass list producing compiled IR. Next, we count the number of instructions of compiled IR and evaluate if the predicted source IR and optimized IR are correct. Since optimized IR could be derived from both generated IR and compiled IR, we save both metrics in the feedback. Additionally, we validate if the predicted IR is compilable, save the error message if any, and calculate the Bleu score between the generated IR and compiled IR.

We compare 3 kinds of feedback (Figure 2). Short Feedback contains predictions and answers for metrics and error messages. Long Feedback contains all derivable metrics and extends Short Feedback by Compiled IR. Since Short and Long Feedback both contain the metrics that come from generated IR, they require a full generation to be constructed. Fast Feedback avoids this by providing only metrics calculated from the pass list and instruction counts. This enables the model to stop generation early, terminating in just a few seconds which is about 10x faster than for other feedback.

When it comes to hardware efficiency, the process of appending Feedback data is extremely efficient. In this moment when the model generates the last output token, the GPU memory contains already prompt and generation. Adding Feedback would just be written in already allocated GPU memory, and the model would be ready for evaluation a second time.

Structuring the Feedback task after prompt and generation has one additional benefit. It reinforces learning of optimization tasks without feedback as well. This happens because the probability of one token depends only on the previous tokens and since we are appending a Feedback task after the task of optimization, it will not have an influence on it. This way we can use the same model for both optimization without feedback and with feedback.

Combining the Feedback approach with sampling can be an effective way of tuning applications. By increasing the temperature in LLM generation the model creates multiple strategies for optimization. Since this process is stochastic there is a higher chance there will be some errors in the generation. Learning the model and how to correct itself could enable it to develop the idea further by fixing itself until it comes to a promising solution.

2.1 Feedback Metrics vs. Model Performance

We found that it is possible to derive a correlation between metrics available in the inference time and a model performance (Figure 3). There is a negative correlation between tgt_inst_cnt_error(C) (the difference between predicted target instruction count and instruction count of IR got by compiling predicted passes) and improvement_over_autotuner. In other words, a smaller error in predicting the target instruction count means that the model is more likely to find a good solution. This is also the case if tgt_IR_BLEU(C) (generated IR compared to compiled IR bleu score) is high and the number of generated flags is large.

To understand more precisely the relation between tgt_inst_cnt_error(C), tgt_IR_BLEU(C), and performance we plot the distribution of their values in Figure 4. When the model correctly predicts the target instruction count it also matches the performance of the autotuner. This means that when we detect this case in the inference, we can stop prompting and accept generated optimization passes. Similarly, we can stop prompting if the compiled and generated

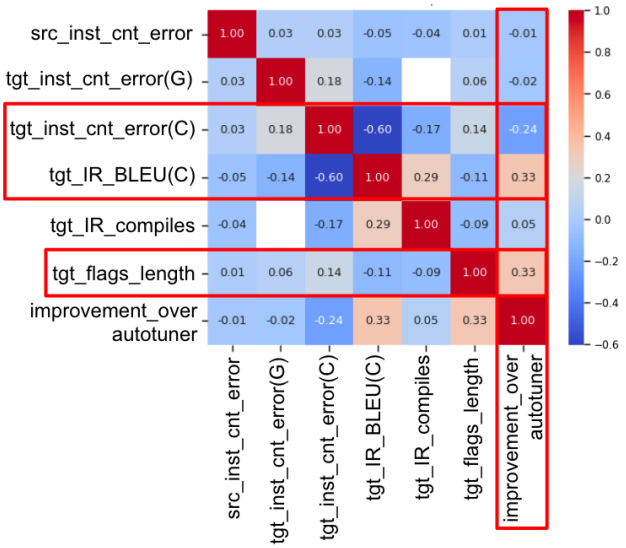


Figure 3. Correlation heatmap of metrics available at inference time. Input and output prompts are described with prefixes (src, tgt). Instruction counts are abbreviated with inst_count. (G) stands for generation while (C) stands for compiled.

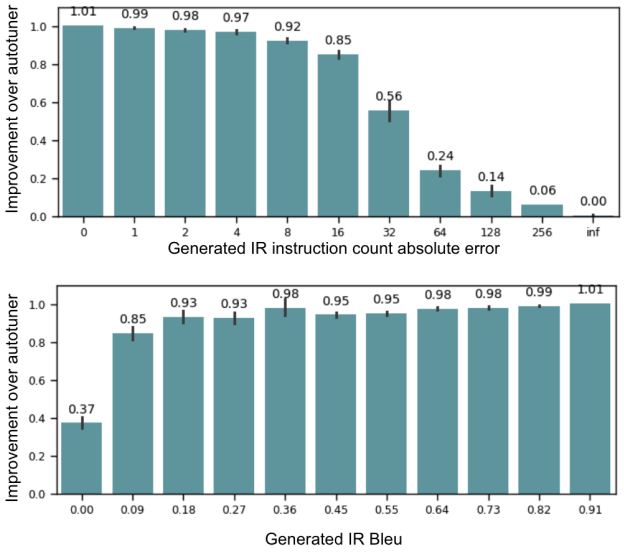


Figure 4. Distribution of absolute error in predicting optimized IR instruction count and Bleu score with respect to performance compared to autotuner.

IR are equal, which results in tgt_IR_BLEU(C) being 1.

3 The Model

We train a 7B-parameter model with LLaMa 2 architecture (Touvron et al., 2023) for each of the Feedback forms. As the starting point for training, we use the best checkpoint from (Cummins et al., 2023) which only predicts the best optimization passes for the given IR. We use the same Byte Pair Encoding (Gage, 1994) tokenizer and model architecture that includes 32 attention heads, 4,096 hidden dimensions, and 32 layers, for a total of 7B parameters.

3.1 Datasets

We construct a training data set for each of the feedback forms. We get feedback data by evaluating the **best model* on 1 million training examples and 100 thousand test examples provided by (Cummins et al., 2023). Additionally, we extract half of the examples from the test set to serve as a validation set.

For all forms of feedback, the prompt will have the structure described in Figure 2. For expected generation, we keep the same format as the original work with an addition of the first line that indicates if the model is sure in its generation. Model outputs *"I am sure!"* if the **best model* correctly predicted target instruction count, which is a strong indication that the model matched the performance of the autotuner. Otherwise, the model outputs *"Let me try again."*.

3.2 Training

We trained all our models for 20,000 steps, with 64 A100 for about 60 GPU days. We use the AdamW optimizer (Loshchilov & Hutter, 2017) with β_1 and β_2 values of 0.9 and 0.95. We use a cosine learning rate schedule with 1,000 warm-up steps, a peak learning rate of $1e{-}5$, and a final learning rate of 1/10th of the peak. We used a batch size of 256 and each batch contains 786,432 tokens for Short and Fast Feedback and 1M tokens for Long Feedback for a total of 16B and 21B tokens respectively. The full training of 20,000 steps made 5.12 iterations over the training corpus.

4 Evaluation

In the evaluation, we are answering the following questions:

- How does the feedback model compare to the original in Task Optimize and Task Feedback?
- How does the feedback model achieve when sampling is enabled?
- Can we use the feedback model to iteratively generate feedback and repair the current solution?

We found that the feedback model keeps the ability to optimize IR even without feedback. When it is allowed to apply two inferences, it can outperform the original model by 0.53% closing the gap to the autotuner by 10%. On the other hand, when the sampling is enabled, we show that the

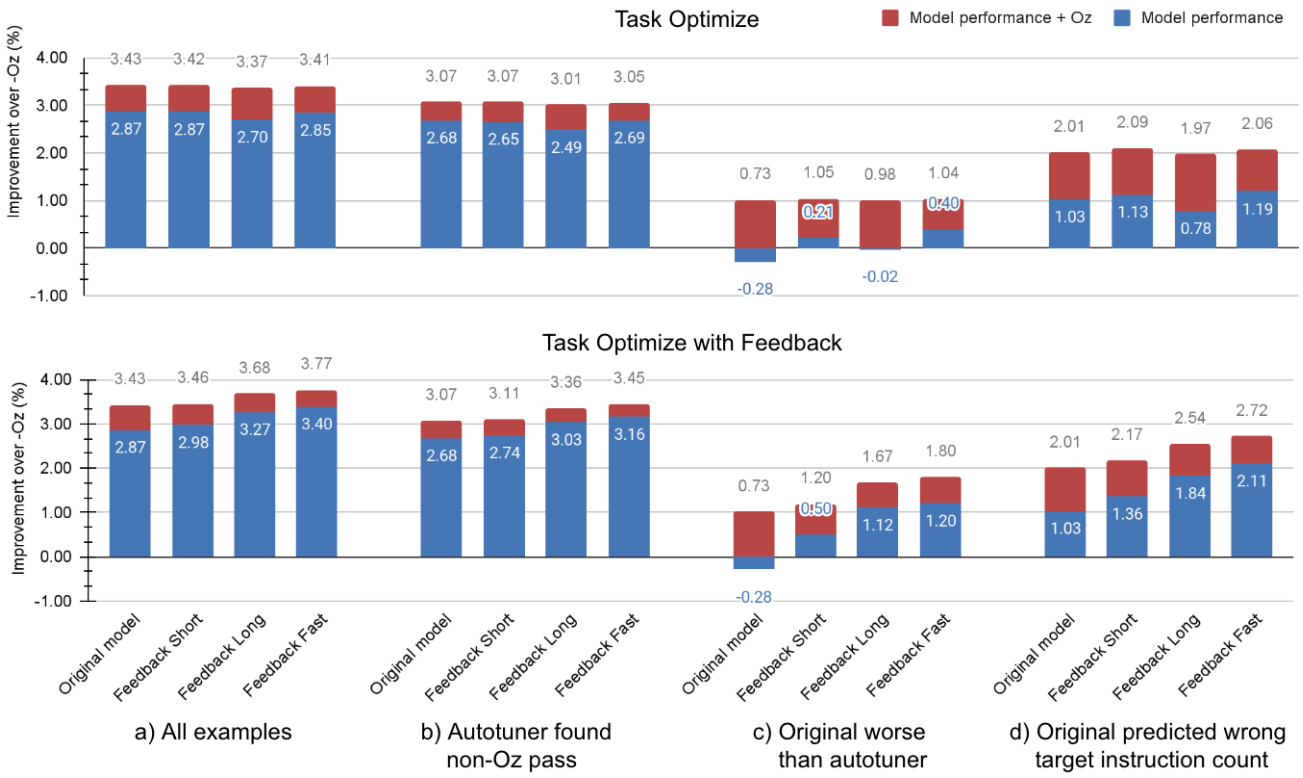


Figure 5. Comparison of the original and feedback models in reducing instruction count. The upper figure shows the performance of Task Optimize. The lower figure shows the performance on Task Feedback, where each model uses their format for feedback. Horizontally, we show the performance on all examples, examples where the autotuner's best pass is non-Oz, examples where the original model was worse than the autotuner, and examples where the original model mispredicted target instruction count. All the models keep the ability to perform Task Optimize while improving the performance when feedback is provided.

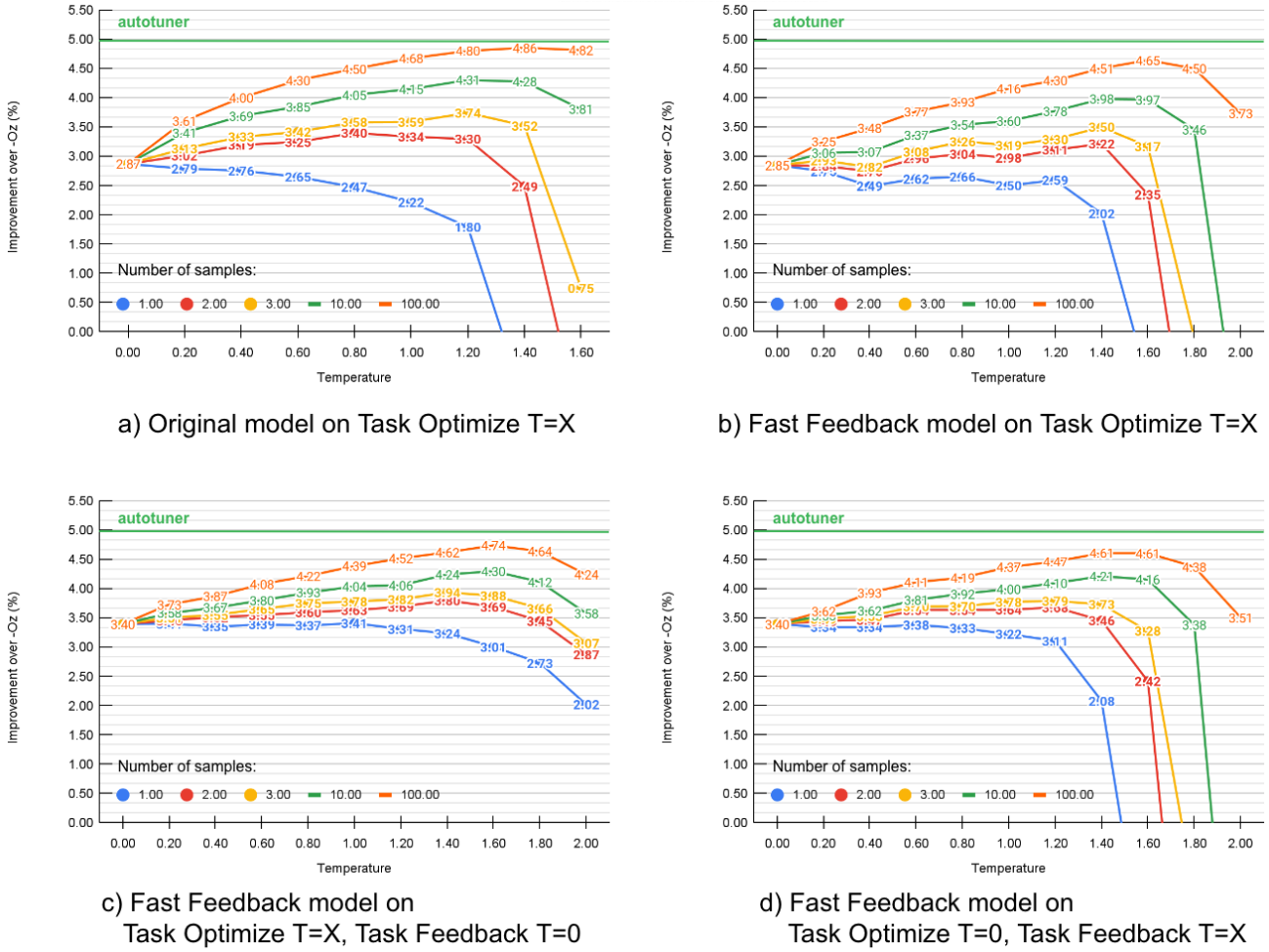
original model achieves up to 98% of the autotuner performance given 100 samples. We evaluate 3 sampling strategies for the feedback model and show that they all fail to match the sampling of the original model. Finally, we compare the performance of the iterative feedback model with the original model given the same amount of computation per sample and we show that with 2 or more samples and a temperature higher than 0.4, the original model outperforms the feedback model.

4.1 How does the feedback model compare to the original in Task Optimize and Task Feedback?

We compare all three Feedback models with the original on the Task Optimize and Task Feedback (Figure 5). In Task Optimize the input prompt consists only of the input IR, while in Task Feedback each model will append the input prompt with the feedback they got from the previous generation in the format defined in Figure 2. Additionally, we show performance on all examples, examples where the autotuner found a non-Oz optimization pass, examples where the original model was worse than the autotuner, and examples where the original model mispredicted the instruction count. Furthermore, we show the performance of the model combined with *-Oz*.

All of the Feedback models perform similarly on average to the original on Task Optimize even without being trained on that task explicitly in the feedback finetuning. Moreover, the feedback models even improved the performance for the examples where the original model performed worse than the autotuner by 0.6% for Fast Feedback. The reason for this is that we add extra information to the input, which enables the model to discriminate hard examples and learn them easier.

In the plot below, we feed the output from Task Optimize to each Feedback model and apply Task Feedback, while keeping the results from Task Optimize for the original model. All the Feedback models improve the performance of the original model by 0.19% with Short Feedback, 0.4% with Long Feedback, and 0.53% for Fast Feedback. Most of the improvement comes from the examples where the



a) Original model on Task Optimize T=X

b) Fast Feedback model on Task Optimize T=X

c) Fast Feedback model on Task Optimize T=X, Task Feedback T=0

d) Fast Feedback model on Task Optimize T=0, Task Feedback T=X

Figure 6. Sampling diagrams of the original and feedback models.

original model performed worse than the autotuner and the examples where the model mispredicted the generated instruction count. Here the Fast Feedback model outperforms the original model by 1.48% and 1.07% respectively.

It is interesting that the Fast Feedback model performs better than Long Feedback, in spite of using a subset of information in the input prompt. We believe the reason for this is that using generated IR in the input prompt introduces noise and makes the model find the relevant information. Since the Fast Feedback model doesn't need to generate IR to create feedback we can iterate with it much faster. For the next experiments, we use the Fast Feedback model for further evaluation.

4.2 Feedback model sampling

We compare the Fast Feedback model against the sampling of the original model with different temperatures (Figure 6). In the first two plots, we show the original and Fast Feedback model on Task Optimize with different temperatures. The original model achieves the peak performance of an astonishing 98% of the autotuner given 100 samples on the temperature 1.4. This result demonstrates the significant power of sampling that opens up interesting directions for future research. With 10 samples the original model achieves about 86% of the autotuner which is a significant win compared to 57% of the temperature 0.

The third plot evaluates the Fast Feedback model starting from Task Optimize with given temperatures and applying Task Feedback with temperature 0. The Fast Feedback model achieves up to 93% on Task Optimize on the temperature 1.6. This is expected since it was not finalized on Task Optimize. Interestingly, it has the peak for the higher temperature, which indicates that its logit distribution is sharper initially and it is more conservative in choosing the next tokens.

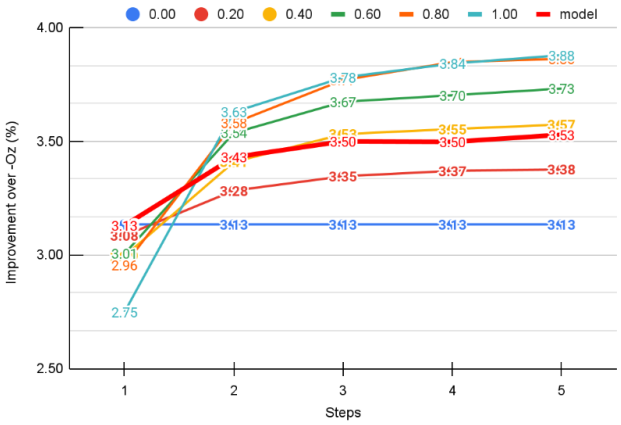


Figure 7. Comparison of the iterative approach (model) versus the sampling of the original model with the same amount of computation. In each step the Fast Feedback model generates feedback for the next step, applying Task Optimize in the first step and Task Feedback afterwards. Once the model outputs "I am sure!" we stop. We allow the same number of generations for the original model.

The fourth plot evaluates the Fast Feedback model starting from Task Optimize on temperature 0 and applies Task Feedback with given temperatures. For the lower number of samples (1, 2, 3) it achieves higher performance than the original model, but it doesn't keep the advantage for 10 or more samples. The reason for this might be that the Fast Feedback model was trained on the feedback of the original model with temperature 0, and the generations become significantly different when the temperature is on. This indicates that training the Feedback model on generations with temperature can increase the performance.

Finally, sampling the Fast Feedback model after the Task Optimize with the temperature 0 fails to deliver performance as well. Similar to the previous method, this method performs well for a smaller number of samples, while it even has lower results than Task Optimize with 100 samples. This might indicate that it generates more conservative solutions than the original model.

5 Additional Experiments

5.1 Feedback model iterative algorithm

In this section, we evaluate the capability of the fast feedback model to iterate based on previous solutions and compare it to the sampling of the original model with the same number of inferences. First, the Fast Feedback model applies Task Optimize, which generates feedback. Then the model applies Task Feedback iteratively by using the feedback of the previous task and generates the feedback for the next step for a total of 5 steps. After every generation, we check if the Fast Feedback model generated *"I am sure!"* and use that as the final result. For the original model we applied Task Optimize and sampled each example as many times as we made steps with the feedback model We use temperatures in the range [0,1] and show the cumulative performance of each approach in Figure 7.

The Fast Feedback model has a similar performance to the original model with temperature 0 on Task Optimize while making the most of performance improvements in the second step (Task Feedback), and slowly increasing by iterating on Task Feedback, achieving performance similar to the original model with temperature 0.4. The original model with temperature 1 starts with the lowest performance, but outperforms all other approaches from step 2. This result demonstrates that for our problem the sampling is more powerful than iteration and should be used instead.

6 Related Work

In recent years we have witnessed tremendous progress in the development of Large Language Models for software engineering. These models are able to generate code and documentation (OpenAI, 2023; Rozière et al., 2023; Chowdhery et al., 2022; Li et al., 2023; Allal et al., 2023; Fried et al., 2023; Gunasekar et al., 2023), translate code between programming languages (Lachaux et al., 2020; Armengol-Estapé & O'Boyle, 2021; Szafraniec et al., 2022), write unit-tests (Ye et al., 2021; Deng et al., 2023; Schäfer et al., 2023), as well as detect and fix bugs (Ahmad et al., 2023; Xia et al., 2023).

The availability of various open-source code datasets (Da Silva et al., 2021; Armengol-Estapé et al., 2022) and the accessibility of platforms like GitHub have enabled models such as CodeLlama (Rozière et al., 2023), ChatGPT (OpenAI), and Codex (Chen et al., 2021) to elevate their coding capabilities. However, it's important to note that these models were not explicitly designed for code optimization. For instance, ChatGPT can perform minor optimizations, like tagging variables for storage in registers, and even attempt more substantial improvements such as vectorization. Nevertheless, it often encounters confusion and makes errors, leading to incorrect code outcomes.

On the other hand, models such as AlphaCode (Li et al., 2022) generate a performant solution by optimizing code on the source level. AlphaCode is fine-tuned on competitive programming problems from the Codeforces platform while using 715.1 GB of code data from GitHub for pretraining. Furthermore, it generates a large corpus of potential solutions from which it chooses the 10 best solutions by implementing sophisticated clustering and filtering mech-

anisms. Similar to Alphacode we demonstrate the power of sampling while targeting compiler-level optimizations instead of the optimizations on the source code.

When it comes to fundamental LLM improvements, Wei et. all (2022a) showed that significant improvement of the LLM can be obtained by splitting answers in step by step manner for models with more than 10B parameters. Brown et. all showed that a few-shot prompting (2020) based on extending prompt with similar (question, answer) pairs additionally increases the performance of the model. Yang et. all (Yang et al., 2023) extend this approach further by iteratively adding generated solutions and their evaluations to the original prompt, together with few-shot prompting. In our approach, we provide more informative feedback based on inference time evaluation of model generations that includes validation of generated pass list, evaluation of predicted instruction counts, and optimized IR.

Finally, the problem of compiler pass ordering has been explored for many decades, as indicated by prior works (Bodin et al., 1998; Kisuki et al., 2000; Fursin et al., 2005). In recent years, machine learning has become an avenue for this optimization task, with various approaches proposed (Wang & O'Boyle, 2018; Leather & Cummins, 2020; Liang et al., 2023; Haj-Ali et al., 2020b; Agakov et al., 2006; Ogilvie et al., 2017). It's noteworthy that the implementation of machine learning in compiler technology extends beyond just optimizing pass order and encompasses a wide range of other challenges (Trofin et al., 2021; Ashouri et al., 2022; Haj-Ali et al., 2020a; Cummins et al., 2017; Phothilimthana et al., 2021). We extend these efforts further by pioneering the use of Large Language Models in compiler optimization.

7 Limitations and Future Work

Sampling has great potential to improve the performance of the original model. In this paper, we saw that the Fast Feedback model fails to improve the performance of the original model when there are larger than 10 samples. This indicates either that the Fast Feedback model doesn't know how to deal with generations on higher temperature, or that it generates always the same output. To mitigate this, we can generate a dataset by sampling the original model and train the Fast feedback model on these data.

Another approach is to disregard the feedback component and implement a smarter sampling heuristic for the original model. For this, we can use beam search and similar techniques when sampling the tokens. AlphaCode (Li et al., 2022) for example generates many possible solutions and then uses clustering and filtering techniques to select the best answers. Developing methods in this area could be more generally applicable in the LLM community.

8 Conclusions

In this paper, we introduce the idea of compiler-generated feedback for Large Language Models. In our approach, the model starts from unoptimized LLVM IR and predicts the best optimization passes and instruction counts and optimized IR. Then we construct feedback on the model's generation and ask the model to try again. We explore 3 formats of feedback with various degrees of information. All forms of the feedback outperform the original model on temperature 0 by 0.11%, 0.4%, and 0.53%.

We further explore the sampling properties of the Fast Feedback and compare it to the original model. The Fast Feedback model fails to improve the performance of the original model when 10 or more samples are given. Additionally, we compare the sampling of the original model to the iterative generation of the Feedback model with the same amount of computing. In this case, we found that sampling of the original model between temperatures [0.4, 1] achieves higher performance.

From this, we conclude that sampling is an irreplaceable technique for getting high performance and more research needs to be done. Potential future directions could be training the Fast Feedback model by collecting the feedback of the original model sampled on the given Temperature. This way Feedback model should become more capable of optimizing more fuzzy generations. Another approach would be to explore further sampling of the original model and develop more sophisticated algorithms that preserve diversity without generating incoherent text.

References

Agakov, F., Bonilla, E., Cavazos, J., Franke, B., Fursin, G., O'Boyle, M., Thomson, J., Toussaint, M., and Williams, C. Using machine learning to focus iterative optimization. In *CGO*, 2006.

Ahmad, B., Thakur, S., Tan, B., Karri, R., and Pearce, H. Fixing hardware security bugs with large language models. *arXiv preprint arXiv:2302.01215*, 2023.

Allal, L. B., Li, R., Kocetkov, D., Mou, C., Akiki, C., Ferrandis, C. M., Muennighoff, N., Mishra, M., Gu, A., Dey, M., Umapathi, L. K., Anderson, C. J., Zi, Y., Poirier, J. L., Schoelkopf, H., Troshin, S., Abulkhanov, D., Romero, M., Lappert, M., Toni, F. D., del Río, B. G., Liu, Q., Bose, S., Bhattacharyya, U., Zhuo, T. Y., Yu, I., Villegas, P., Zocca, M., Mangrulkar, S., Lansky, D., Nguyen, H., Contractor, D., Villa, L., Li, J., Bahdanau, D., Jernite, Y., Hughes, S., Fried, D., Guha, A., de Vries, H., and von Werra, L. SantaCoder: don't reach for the stars! *arXiv:2301.03988*, 2023.

Armengol-Estapé, J. and O'Boyle, M. F. Learning C to

x86 Translation: An Experiment in Neural Compilation. *arXiv:2108.07639*, 2021.

Armengol-Estapé, J., Woodruff, J., Brauckmann, A., Magalhães, J. W. d. S., and O'Boyle, M. F. Exebench: an ml-scale dataset of executable c functions. In *Proceedings of the 6th ACM SIGPLAN International Symposium on Machine Programming*, pp. 50–59, 2022.

Ashouri, A. H., Elhoushi, M., Hua, Y., Wang, X., Manzoor, M. A., Chan, B., and Gao, Y. MLGOPerf: An ML Guided Inliner to Optimize Performance. *arXiv:2207.08389*, 2022.

Bodin, F., Kisuki, T., Knijnenburg, P., O'Boyle, M., and Rohou, E. Iterative compilation in a non-linear optimisation space. In *FDO*, 1998.

Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33: 1877–1901, 2020.

Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H. P. d. O., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., Ray, A., Puri, R., Krueger, G., Petrov, M., Khlaaf, H., Sastry, G., Mishkin, P., Chan, B., Gray, S., Ryder, N., Pavlov, M., Power, A., Kaiser, L., Bavarian, M., Winter, C., Tillet, P., Such, F. P., Cummings, D., Plappert, M., Chantzis, F., Barnes, E., Herbert-Voss, A., Guss, W. H., Nichol, A., Paino, A., Tezak, N., Tang, J., Babuschkin, I., Balaji, S., Jain, S., Saunders, W., Hesse, C., Carr, A. N., Leike, J., Achiam, J., Misra, V., Morikawa, E., Radford, A., Knight, M., Brundage, M., Murati, M., Mayer, K., Welinder, P., McGrew, B., Amodei, D., McCandlish, S., Sutskever, I., and Zaremba, W. Evaluating Large Language Models Trained on Code. *arXiv:2107.03374*, 2021.

Chowdhery, A., Narang, S., Devlin, J., Bosma, M., Mishra, G., Roberts, A., Barham, P., Chung, H. W., Sutton, C., Gehrmann, S., Schuh, P., Shi, K., Tsvyashchenko, S., Maynez, J., Rao, A., Barnes, P., Tay, Y., Shazeer, N., Prabhakaran, V., Reif, E., Du, N., Hutchinson, B., Pope, R., Bradbury, J., Austin, J., Isard, M., Gur-Ari, G., Yin, P., Duke, T., Levskaya, A., Ghemawat, S., Dev, S., Michalewski, H., Garcia, X., Misra, V., Robinson, K., Fedus, L., Zhou, D., Ippolito, D., Luan, D., Lim, H., Zoph, B., Spiridonov, A., Sepassi, R., Dohan, D., Agrawal, S., Omernick, M., Dai, A. M., Pillai, T. S., Pellat, M., Lewkowycz, A., Moreira, E., Child, R., Polozov, O., Lee, K., Zhou, Z., Wang, X., Saeta, B., Diaz, M., Firat, O., Catasta, M., Wei, J., Meier-Hellstern, K., Eck, D., Dean, J., Petrov, S., and Fiedel, N. PaLM: Scaling Language Modeling with Pathways. *arXiv:2204.02311*, 2022.

Cummins, C., Petoumenos, P., Wang, Z., and Leather, H. End-to-End Deep Learning of Optimization Heuristics. In *PACT*, 2017.

Cummins, C., Fisches, Z., Ben-Nun, T., Hoefler, T., O'Boyle, M., and Leather, H. ProGraML: A Graph-based Program Representation for Data Flow Analysis and Compiler Optimizations. In *ICML*, 2021.

Cummins, C., Seeker, V., Grubisic, D., Elhoushi, M., Liang, Y., Roziere, B., Gehring, J., Gloeckle, F., Hazelwood, K., Synnaeve, G., et al. Large language models for compiler optimization. *arXiv preprint arXiv:2309.07062*, 2023.

Da Silva, A. F., Kind, B. C., de Souza Magalhães, J. W., Rocha, J. N., Guimaraes, B. C. F., and Pereira, F. M. Q. Anghabench: A suite with one million compilable c benchmarks for code-size reduction. In *2021 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, pp. 378–390. IEEE, 2021.

Deng, Y., Xia, C. S., Peng, H., Yang, C., and Zhang, L. Large language models are zero-shot fuzzers: Fuzzing deep-learning libraries via large language models. In *ISSTA*, 2023.

Fried, D., Aghajanyan, A., Lin, J., Wang, S., Wallace, E., Shi, F., Zhong, R., Yih, W.-t., Zettlemoyer, L., and Lewis, M. InCoder: A Generative Model for Code Infilling and Synthesis. *arXiv:2204.05999*, 2023.

Fursin, G. G., O'Boyle, M. F. P., and Knijnenburg, P. M. W. Evaluating Iterative Compilation. In *LCPC*, 2005.

Gage, P. A New Algorithm for Data Compression. *C Users Journal*, 12(2), 1994.

Gunasekar, S., Zhang, Y., Aneja, J., Mendes, C. C. T., Giorno, A. D., Gopi, S., Javaheripi, M., Kauffmann, P., de Rosa, G., Saarikivi, O., Salim, A., Shah, S., Behl, H. S., Wang, X., Bubeck, S., Eldan, R., Kalai, A. T., Lee, Y. T., and Li, Y. Textbooks Are All You Need. *arXiv:2306.11644*, 2023.

Haj-Ali, A., Ahmed, N. K., Willke, T., Shao, S., Asanovic, K., and Stoica, I. Neurovectorizer: End-to-end vectorization with deep reinforcement learning. In *CGO*, 2020a.

Haj-Ali, A., Huang, Q., Moses, W., Xiang, J., Wawrzynek, J., Asanovic, K., and Stoica, I. AutoPhase: Juggling HLS Phase Orderings in Random Forests with Deep Reinforcement Learning. In *MLSys*, 2020b.

Kisuki, T., Knijnenburg, P., and O'Boyle, M. Combined selection of tile sizes and unroll factors using iterative compilation. In *PACT*, 2000.

Lachaux, M.-A., Roziere, B., Chanussot, L., and Lample, G. Unsupervised Translation of Programming Languages. *arXiv:2006.03511*, 2020.

Leather, H. and Cummins, C. Machine Learning in Compilers: Past, Present and Future. In *FDL*, 2020.

Li, R., Allal, L. B., Zi, Y., Muennighoff, N., Kocetkov, D., Mou, C., Marone, M., Akiki, C., Li, J., Chim, J., Liu, Q., Zheltonozhskii, E., Zhuo, T. Y., Wang, T., Dehaene, O., Davaadorj, M., Lamy-Poirier, J., Monteiro, J., Shliazhko, O., Gontier, N., Meade, N., Zebaze, A., Yee, M.-H., Umapathi, L. K., Zhu, J., Lipkin, B., Oblokulov, M., Wang, Z., Murthy, R., Stillerman, J., Patel, S. S., Abulkhanov, D., Zocca, M., Dey, M., Zhang, Z., Fahmy, N., Bhattacharyya, U., Yu, W., Singh, S., Luccioni, S., Villegas, P., Kunakov, M., Zhdanov, F., Romero, M., Lee, T., Timor, N., Ding, J., Schlesinger, C., Schoelkopf, H., Ebert, J., Dao, T., Mishra, M., Gu, A., Robinson, J., Anderson, C. J., Dolan-Gavitt, B., Contractor, D., Reddy, S., Fried, D., Bahdanau, D., Jernite, Y., Ferrandis, C. M., Hughes, S., Wolf, T., Guha, A., von Werra, L., and de Vries, H. StarCoder: may the source be with you! *arXiv:2305.06161*, 2023.

Li, Y., Choi, D., Chung, J., Kushman, N., Schrittwieser, J., Leblond, R., Eccles, T., Keeling, J., Gimeno, F., Lago, A. D., Hubert, T., Choy, P., d'Autume, C. d. M., Babuschkin, I., Chen, X., Huang, P.-S., Welbl, J., Gowal, S., Cherepanov, A., Molloy, J., Mankowitz, D. J., Robson, E. S., Kohli, P., de Freitas, N., Kavukcuoglu, K., and Vinyals, O. Competition-level code generation with AlphaCode. *Science*, 378(6624), 2022.

Liang, Y., Stone, K., Shameli, A., Cummins, C., Elhoushi, M., Guo, J., Steiner, B., Yang, X., Xie, P., Leather, H., and Tian, Y. Learning Compiler Pass Orders using Coreset and Normalized Value Prediction. In *ICML*, 2023.

Loshchilov, I. and Hutter, F. Decoupled Weight Decay Regularization. *arXiv:1711.05101*, 2017.

Ogilvie, W. F., Petoumenos, P., Wang, Z., and Leather, H. Minimizing the cost of iterative compilation with active learning. In *CGO*, 2017.

OpenAI. ChatGPT. https://chat.openai.com/.

OpenAI. GPT-4 Technical Report. *arXiv:2303.08774*, 2023.

Papineni, K., Roukos, S., Ward, T., and Zhu, W.-J. BLEU: A Method for Automatic Evaluation of Machine Translation. In *ACL*, 2002.

Phothilimthana, P. M., Sabne, A., Sarda, N., Murthy, K. S., Zhou, Y., Angermueller, C., Burrows, M., Roy, S., Mandke, K., Farahani, R., et al. A Flexible Approach to Autotuning Multi-pass Machine Learning Compilers. In *PACT*, 2021.

Rozière, B., Gehring, J., Gloeckle, F., Sootla, S., Gat, I., Tan, X. E., Adi, Y., Liu, J., Remez, T., Rapin, J., Kozhevnikov, A., Evtimov, I., Bitton, J., Bhatt, M., Ferrer, C. C., Grattafiori, A., Xiong, W., Défossez, A., Copet, J., Azhar, F., Touvron, H., Martin, L., Usunier, N., Scialom, T., and Synnaeve, G. Code Llama: Open Foundation Models for Code. *arXiv:2308.12950*, 2023.

Schäfer, M., Nadi, S., Eghbali, A., and Tip, F. Adaptive Test Generation Using a Large Language Model. *arXiv:2302.06527*, 2023.

Szafraniec, M., Roziere, B., Charton, F., Leather, H., Labatut, P., and Synnaeve, G. Code Translation with Compiler Representations. *arXiv:2207.03578*, 2022.

Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., Babaei, Y., Bashlykov, N., Batra, S., Bhargava, P., Bhosale, S., et al. Llama 2: Open Foundation and Fine-Tuned Chat Models. *arXiv:2307.09288*, 2023.

Trofin, M., Qian, Y., Brevdo, E., Lin, Z., Choromanski, K., and Li, D. MLGO: a Machine Learning Guided Compiler Optimizations Framework. *arXiv:2101.04808*, 2021.

Wang, Z. and O'Boyle, M. Machine Learning in Compiler Optimisation. *arXiv:1805.03441*, 2018.

Wei, J., Wang, X., Schuurmans, D., Bosma, M., Xia, F., Chi, E., Le, Q. V., Zhou, D., et al. Chain-of-thought prompting elicits reasoning in large language models. In *NeurIPS*, 2022a.

Wei, J., Wang, X., Schuurmans, D., Bosma, M., Xia, F., Chi, E., Le, Q. V., Zhou, D., et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, 35: 24824–24837, 2022b.

Xia, C. S., Wei, Y., and Zhang, L. Automated program repair in the era of large pre-trained language models. In *Proceedings of the 45th International Conference on Software Engineering (ICSE 2023). Association for Computing Machinery*, 2023.

Yang, C., Wang, X., Lu, Y., Liu, H., Le, Q. V., Zhou, D., and Chen, X. Large language models as optimizers. *arXiv preprint arXiv:2309.03409*, 2023.

Ye, G., Tang, Z., Tan, S. H., Huang, S., Fang, D., Sun, X., Bian, L., Wang, H., and Wang, Z. Automated conformance testing for JavaScript engines via deep compiler fuzzing. In *PLDI*, 2021.